

ROS2 – Humble

Writer

Mehmet Özkaya

Installation

Go to the ROS website and follow the instructions.

After the Installation

After the installation, we can check if ROS setup is ready by opening a new terminal and write:

- `source/opt/ros/humble/setup.bash`
- `ros2`

If we see some instruction below the command, it means ROS works properly.

Running a Package

If we want to run a package and a node in ROS2, we use this command:

- `ros2 run <package_name> <node_name>`

Package

Package is a kind of library that contains many nodes.

Node

Node is an operation that we want to do. It can be a code that move a robot.

For ex:

We can open 2 different terminals and for the first one we can run a talker Node by:

- `ros2 run demo_nodes_cpp talker`

And for the second one we can run a listener Node by:

- `ros2 run demo_nodes_cpp listener`

Rqt Graph

It is a kind of tool to see the graph of the interaction. Graph is the representation of the nodes that are running.

We can see the rqt graph by:

- `rqt_graph`



Talker : Sends information

Listener : Takes the information sent from talker

Turtlesim

To see an example in a simulation, we can use some built-in simulations in ROS2 by:

- `ros2 run turtlesim turtlesim_node`

It will open a window which contains a turtle in middle. This is our listener Node. To send information to the listener, we open a new terminal and create the talker by:

- `ros2 run turtlesim turtle_teleop_key`

After that, it will listen to our keyboard to move the turtle.

ROS2 Workspace

ROS2 Workspace is to organize our codes and make our files more systematic. To install our ROS2 workspace, we should first install colcon. Colcon is the build tool for ROS2. To install Colcon:

- `sudo apt update`
- `sudo apt install python3-colcon-common-extensions`

Now we can use Colcon to download ROS2 Workspace. To enable the autocompletion:

- `cd /usr/share/colcon_argcomplete/hook/`
- `gedit ~/.bashrc`

It will open a file and at the end of the line we can add:

- `source /opt/ros/humble/setup.bash` -> to run ros2 directly in the terminal
- `source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash` -> to enable auto complete

To download our ROS2 Workspace:

- `mkdir ros2_ws` -> ROS2 Workspace folder in our base path
- `cd ros2_ws` -> Move to ros2_ws file
- `mkdir src` -> Create a source folder. Source folder contains the nodes of the package.
- `colcon build` -> To build our package by colcon. After this operation we should have 3 new folders in the current folder.
- `cd install/` -> move install folder

Now we should source our setup.bash file created by colcon. To do it:

- `gedit .bashrc` -> It will open a file.

Add the code below at the end of the file:

- `source ~/ros2_ws/install/setup.bash`

Now we can easily run our Nodes.

Creating a ROS2 Python Package

To create a node, first we need to create a package. To create a package:

- `cd ros2_ws/` -> To move workspace
- `cd src` -> To move source file. We should create the package here.
- `ros2 pkg create <name of the package> --build-type ament_python /ament_cmake --dependencies rclpy` (For ex: `ros2 pkg create my_robot_controller ament_python`) to create the Nodes with Python with specified dependency rclpy library.

To navigate into the created package we can use a code editor. I will use VSCode.

Go to the src directory and run:

- `code .`

In this file:

- **package.xml** : This file contains the name of the package, version, description and the licence. Also the dependency of the package and the build_type.
- **setup.cfg** : Dont touch this file.
- **setup.py** : It is similar to the package.xml.
- **<package_name> folder** : We are going to create our nodes.

To build this package, we should move to the workspace directory and run:

- `colcon build`

This command will fetch all of the nodes in our package and install them in our install folder. After this operation we can run our nodes in our terminal. Now we can write our nodes.

Creating a Node with Python and OOP

- `cd ros2_ws` -> move into ros2 workspace
- `cd src`
- `cd <package_name>`
- `cd <package_name>`
- `touch <node_name>.py` -> create a python file to write our node
- `chmod +x <node_name>.py` -> to make it executable
- Move into src dir again
- `code .` -> open VSCode

At the beginning of the file, add this command to determine it is using python3 as interpreter

- `#!/usr/bin/env python3`

To run this node from the terminal, we should create a main function and add these lines:

```
if __name__ == "__main__":  
    main()
```

The first thing we should do is creating the communication in the main function:

```
def main(args = None):  
    rclpy.init(args = args)  
    # Everything  
    rclpy.shutdown()
```

Let's create our first node outside of the main func. It inherit from the Node from `rclpy.node` :

```
class MyNode(Node):  
    def __init__(self):  
        super().__init__("first_node")  
        self.get_logger().info("Hello from ROS2")
```

And create a node in the main func:

```
node = MyNode()
```

Now we can run our node in the terminal by (in the dir where our node file exists):

- `./my_first_node.py`

If we want our node run continuously:

```
node = MyNode()  
rclpy.spin(node)
```

- spin(node) method keeps the node alive.

We ran our code as a python script but we want to run it using ros2 functionalities. To do it in setup.py file:

```
entry_points={
    'console_scripts': [
        "test_node = <package_name>.<node_name>:main"
    ],
}
```

To install our Node, move to workspace dir:

- `colcon build`

After we built the Node, we have to source our workspace:

- `source ~/.bashrc`

Now we can run our node such as:

- `ros2 run my_robot_controller test_node` -> be careful that in setup.py file, we named it as test_node.

Everytime we change our code, we have to colcon build the Nodes as. After this operation we dont need to build the code again and again, changes are automatically applied:

- `colcon build --symlink-install`

And then again:

- `source ~/.bashrc`

Run the node again:

- `ros2 run my_robot_controller test_node`

Now let's create timer and a callback inside the node:

We create a timer to do the same operation (callback) with specific time zone:

```
class MyNode(Node):
    def __init__(self):
        super().__init__("first_node")
        self.create_timer(1.0, self.timer_callback)

    def timer_callback(self):
        self.get_logger().info("Hello")
```

We have created a talker Node, now we need a listener Node to take the information.

We can check all nodes by (we must run the code while running the command):

- `ros2 node list` -> returns the node list
- `ros2 node info <node name>` -> get some information about the node.

ROS2 Topic

ROS2 Topic is about the communication between the nodes in the package.



Here /chatter is the topic. As we can see it is the communication between the talker and the listener nodes.

Talker is going to publish into chatter topic and listener is subscribed to the chatter topic.

Talker sends information and listener takes this information through the chatter topic.

We can see the topics by running the command (we have to run the nodes while executing it):

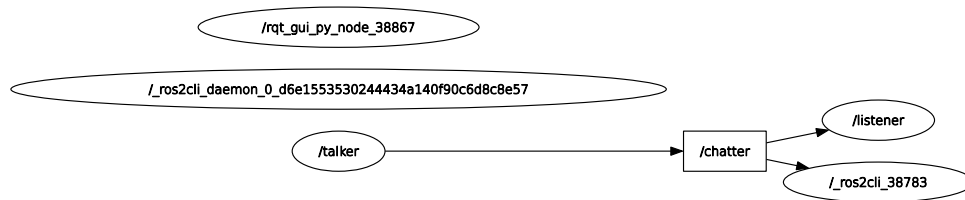
- `ros2 topic list` -> returns the names of the topics.
- `ros2 topic info <topic name>` -> get the information about the topic such as:
 - Type: std_msgs/msg/String
 - Publisher count: 1
 - Subscription count: 1

- **Type** : type of the information
- **Publisher count** : publisher count
- **Subscriber count** : subscriber count

We can get more knowledge about the information by:

- `ros2 interface show std_msgs/msg/String`

A topic can have many publisher and subscribers:



As you can see here, we have 1 publisher named talker and 2 subscribers listener and another one.

Also for the turtlesim package, we have the topic cmd_vel. If we investigate the data sent:

- `ros2 interface show geometry_msgs/msg/Twist`

This expresses velocity in free space broken into its linear and angular parts.

Vector3 linear

float64 x

float64 y

float64 z

Vector3 angular

float64 x

float64 y

float64 z

Here we sent an information with 6 dimension value.

Creating a Publisher from scratch

1. Move into ros2 workspace and src folder: `cd ros2_ws/src/`
2. Move into your python files directory: `cd <package_name>/<package_name>`
3. Create a new python file: `touch <node_name>.py`
4. Make the python file executable: `chmod +x <filename>.py`
5. Move back into src dir: `cd ../../`
6. Open in VSCode: `code .`

Now we can write our Node:

! Be careful that each package that we are going to use should be added in package.xml as:

- `<depend><package_name></depend>`

To create a publisher:

```
class DrawCircleNode(Node):
    def __init__(self):
        super().__init__("draw_circle")
        self.cmd_vel_pub = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
        self.get_logger().info("Draw circle node has been started!")
```

In `create_publisher` method: `create_publisher(<message type>, <topic name>, <queue size>)`

Now let's write a callback function and a timer. Also adjust the message to send and then send the message:

```
class DrawCircleNode(Node):
    def __init__(self):
        super().__init__("draw_circle")
        self.cmd_vel_pub = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
        self.timer = self.create_timer(0.5, self.send_velocity_command)
        self.get_logger().info("Draw circle node has been started!")

    def send_velocity_command(self):
        msg = Twist()
        msg.linear.x = 2.0
        msg.angular.z = 1.0

        # To publish the msg
        self.cmd_vel_pub.publish(msg)
```

Now we should install the node in `setup.py` file:

1. Move back to `ros2` workspace: `cd ../..`
2. Built the package again: `colcon build --symlink-install` (symlink only works for the specific file. So if you create a new file in the package, you should run `colcon build`)
3. Move back to root and run this command to setup `ros2` workspace env.
4. Run the node by: `ros2 run <package name> <node name>`

Creating a Subscriber from scratch

1. First we have know what is the message type of the topic that we want to subscribe:
 - `ros2 topic list`: it shows the topic list.
 - `ros2 topic info <topic name>` and here we can see the message type of the topic we want to subscribe.
2. Move into the Python nodes files directory: `cd ../..`
3. Create a new node python file: `touch <filename>.py`
4. Make the file executable: `chmod +x <filename>.py`
5. Create the subscriber Node as follows:

```
class PoseSubscriberNode(Node):
    def __init__(self):
        super().__init__("pose_subscriber")
```

```
self.pose_subscriber = self.create_subscription(Pose, "/turtle1/pose",
self.pose_callback, 10)
```

```
def pose_callback(self, msg: Pose):
    self.get_logger().info(str(msg))
```

6. We created a callback function to do some operation in the received message. Now let's install the Node by adding another node in setup.py such as:

```
"pose_subscriber = my_robot_controller.pose_subscriber:main"
```

In the terminal:

7. Move into the root dir: `cd ../..`
8. Build the package again (ros2_ws): `colcon build --symlink-install`
9. Setup the env again: `source ~/.bashrc`
10. Run the Node: `ros2 run <package name> <node name>`

Creating a Loop System with Publisher and Subscriber

1. Move into the python files directory: `cd ../..`
2. Create a new node python file: `touch <node name>.py`
3. Make the file executable: `chmod +x <filename>.py`
4. Move back into src folder: `cd ../..`
5. Open in VSCode by: `code .`
6. To create a closed loop system:

```
class TurtleController(Node):
    def __init__(self):
        super().__init__("turtle_controller")
        self.cmd_vel_publisher = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
        self.pose_subscriber = self.create_subscription(Pose, "/turtle1/pose",
self.pose_callback, 10)
        self.get_logger().info("Turtle controller has been started.")
```

```
def pose_callback(self, pose: Pose):
    cmd = Twist()
    if pose.x > 9.0 or pose.x < 2.0 or pose.y > 9.0 or pose.y < 2.0:
        cmd.linear.x = 1.0
        cmd.angular.z = 0.9
    else:
        cmd.linear.x = 5.0
        cmd.angular.z = 0.0
    self.cmd_vel_publisher.publish(cmd)
```

7. Install the node in setup.py, add this line:
`"turtle_controller = my_robot_controller.turtle_controller:main"`
8. Build the package again: `colcon build --symlink-install`
9. Setup the env again: `ros2 run my_robot_controller turtle_controller`

ROS2 Services

We can construct a client-server interaction with ROS2 Services. We can see the services list by:

- `ros2 service list`

We can get more information of a service by:

- `ros2 service type <service name>` and
- `ros2 interface show example_interfaces/srv/AddTwoInts` -> We are going to have an output like:

```
int64 a
int64 b
---
int64 sum
```

Here 'a' and 'b' are the inputs or the requests and 'sum' is the response.

How can we send a request:

1-) Start the server to wait for a request: `ros2 run demo_nodes_cpp add_two_ints_server`

2-) In another terminal: `ros2 service call <service name> <service type> <arguments>` (for example: `ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a:2, 'b':5}"`)

Then we can get a response such as:

response:

```
example_interfaces.srv.AddTwoInts_Response(sum=7)
```

This is one the types of services. We get a response with respect to the computation of the request. The other type is about the configuration of something. Let's say we have a robot with many configuration. We can send a request to a ROS2 server to change the configuration of the robot.

Let's investigate in an example:

1. Start turtlesim: `ros2 run turtlesim turtlesim_node`
2. Let's see some example configurations for turtlesim: `ros2 service list`
Here for example:
 - `/clear` -> clear the path
 - `/kill` -> to kill the turtle
 - `/reset` -> To reset
 - `/spawn` -> To spawn another turtle
3. Let's try to change the path's color:
 - a) First we should learn the service type: `ros2 service type /turtle1/set_pen`
 - b) Let's learn more about the service info: `ros2 interface show turtlesim/srv/SetPen`

- c) Now we can send a request to the service: `ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r':256, 'g':0, 'b':0, 'width':3, 'off':0}"`

Writing a ROS2 Service Client with Python

Let's make some changes in our code.

```
class TurtleController(Node):
    def __init__(self):
        super().__init__("turtle_controller")
        self.cmd_vel_publisher = self.create_publisher(Twist, "/turtle1/cmd_vel", 10)
        self.pose_subscriber = self.create_subscription(Pose, "/turtle1/pose",
self.pose_callback, 10)
        self.get_logger().info("Turtle controller has been started.")

    def pose_callback(self, pose: Pose):
        cmd = Twist()
        if pose.x > 9.0 or pose.x < 2.0 or pose.y > 9.0 or pose.y < 2.0:
            cmd.linear.x = 1.0
            cmd.angular.z = 0.9
        else:
            cmd.linear.x = 5.0
            cmd.angular.z = 0.0

        self.cmd_vel_publisher.publish(cmd)
        if pose.x > 5.5:
            self.call_set_pen_service(255, 0, 2, 3, 0)
        else:
            self.call_set_pen_service(0, 255, 0, 3, 0)

    def call_set_pen_service(self, r, g, b, width, off):
        client = self.create_client(SetPen, "/turtle1/set_pen")
        while not client.wait_for_service(1.0):
            self.get_logger().warn("Waiting for service...")

        request = SetPen.Request()
        request.r = r
        request.g = g
        request.b = b
        request.width = width
        request.off = off

        future = client.call_async(request)
        future.add_done_callback(partial(self.callback_set_pen))

    def callback_set_pen(self, future):
```

```
try:  
    response = future.result()  
except Exception as e:  
    self.get_logger().error("Service call failed %r" % (e,))
```

Let's reduce the calling rate for the service. We have to adjust the frequency. For example we don't have to call this service 10 times in a second. We can call it when we need. Let's learn how frequently we get the pose:

- `ros2 topic hz <topic name>` (It must be running.) (For example: `ros2 topic hz /turtle1/pose`)

average rate: 62.500

min: 0.014s max: 0.018s std dev: 0.00055s window: 4222

It means this topic is called 62.5 times in a second